

关系数据理论

1.问题的提出

1.1 关系模式和第一范式

关系模式通常是一个五元组： $R(U, D, DOM, F)$ ，但是D,DOM和模式设计关系不大，可以简化为 $R(U, F)$ ，U是属性名集合，F是一组数据依赖。

如果满足**每个分量都是不可分的数据项**，就属于第一范式（1NF）。

1.2 数据依赖

数据依赖是一个关系内部**属性与属性**之间的一种约束关系。通过属性间值的相等与否体现出来的数据间相关联系。

类型：**函数依赖（FD）**，**多值依赖（MVD）**。

1.3 好的关系模式

一个好的关系模式不会发生**插入异常、删除异常和更新异常**，**数据冗余应尽可能少**。不好的模式设计，是因为存在不好的数据依赖。

1.4 关系的规范化

范式：**是对一个关系中允许存在的数据依赖的要求**。

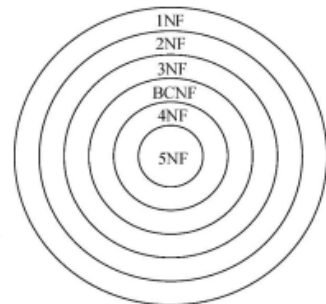
根据对数据依赖要求程度的不同，分为不同级别的范式。从低到高分为：
1NF,2NF,3NF,BCNF,4NF,5NF。

范式可以理解为是**符合某一种级别的关系模式的集合**。

□ 各种范式之间的联系

$$1NF \supset 2NF \supset 3NF \supset BCNF \supset 4NF \supset 5NF$$

- 如果一个关系模式满足高级别范式对数据依赖的要求，那么它肯定也满足低级别范式对函数依赖的要求；
- 一个关系模式满足低级别范式对数据依赖的要求，但它不一定能满足高级别范式对函数依赖的要求。
- 如果某一关系模式R满足范式n的要求，则称关系模式R为第n范式，可简记为 $R \in nNF$
- 一个低一级范式的关系模式，通过**模式分解（schema decomposition）**可以转换为若干个高一级范式的关系模式的集合，这种过程就叫**规范化（normalization）**。



2.规范化

2.1 函数依赖

设 $R(U)$ 是一个属性集 U 上的关系模式， X 和 Y 是 U 的子集。若对于 $R(U)$ 的任意一个可能的关系 r ， r 中不可能存在两个元组在 X 上的属性值相等，而在 Y 上的属性值不等，则称“ X 函数确定 Y ”或“ Y 函数依赖于 X ”，记作 $X \rightarrow Y$ 。 X 称为这个函数依赖的决定因素， Y 称为这个函数依赖的依赖因素。

若 $X \rightarrow Y$ 且 $Y \rightarrow X$ ，则记为 $X \leftrightarrow Y$ 。

若 Y 不函数依赖于 X ，则记为 $X \nrightarrow Y$ 。

- 函数依赖不是指关系模式 R 的某个或某些关系实例满足的约束条件，而是指 R 的所有关系实例均要满足的约束条件
- 函数依赖是语义范畴的概念，只能根据数据的语义来确定函数依赖
- 数据库设计者可以根据现实世界中的语义约束作强制的规定

□ [例] 学生关系 $Student(Sno, Sname, Ssex, Sage, Sdept)$

➤ 假设不允许学生重名，则有：

$Sno \rightarrow Ssex$
 $Sno \rightarrow Sage$
 $Sno \rightarrow Sdept$
 $Sno \leftrightarrow Sname$
 $Sname \rightarrow Ssex$
 $Sname \rightarrow Sage$
 $Sname \rightarrow Sdept$

- Sno 是学生的码
- 学生不允许同名
- 每个学生只能就读于一个院系

➤ 但是

$Ssex \nrightarrow Sage$
 $Ssex \nrightarrow Sdept$

2.2 完全函数依赖vs部分函数依赖

在关系模式 $R(U)$ 中，对于 U 的子集 X 和 Y ，

$X \rightarrow Y$ 但 $Y \not\subseteq X$ ，则称 $X \rightarrow Y$ 是**非平凡函数依赖**

$X \rightarrow Y$ 但 $Y \subseteq X$ ，则称 $X \rightarrow Y$ 是**平凡函数依赖**

- 对于任一关系模式，平凡函数依赖都是必然成立的，它不反映新的语义（映射到自身或子集）

□ 例：在关系 $SC(Sno, Cno, Grade)$ 中，

➤ 有非平凡函数依赖： $(Sno, Cno) \rightarrow Grade$

➤ 有平凡函数依赖：
 $(Sno, Cno) \rightarrow Sno$
 $(Sno, Cno) \rightarrow Cno$

在 $R(U)$ 中，如果 $X \rightarrow Y$ ，并且对于 X 的任何一个真子集 X' ，都有 $X' \nrightarrow Y$ ，则称 Y 对 X **完全函数依赖**，记作 $X \xrightarrow{F} Y$

推论：单一决定因素一定是完全函数依赖

若 $X \rightarrow Y$ ，但 Y 不完全函数依赖于 X ，则称 Y 对 X **部分函数依赖**，记作 $X \xrightarrow{P} Y$

□ 由于： $Sno \nrightarrow Grade, Cno \nrightarrow Grade, Sno \rightarrow Sdept$

因此： $(Sno, Cno) \xrightarrow{F} Grade$

$(Sno, Cno) \xrightarrow{P} Sdept$

2.3 传递函数依赖vs直接函数依赖

在 $R(U)$ 中，如果 $X \rightarrow Y (Y \not\subseteq X), Y \nrightarrow X, Y \rightarrow Z, Z \not\subseteq Y$ ，则称 Z 对 X **传递函数依赖**。记为：
 $X \xrightarrow{\text{传递}(t)} Z$ 。

□ [例] 在关系 $Std(Sno, Sdept, Mname)$ 中

➤ 有： $Sno \rightarrow Sdept, Sdept \nrightarrow Sno, Sdept \rightarrow Mname$

➤ 所以， $Sno \xrightarrow{\text{传递}} Mname$

□ [例] 在学生关系 $Student(Sno, Sname, Ssex, Sage, Sdept)$ 中

➤ 有： $Sno \leftrightarrow Sname$

➤ 所以， $Sno \rightarrow Sdept$ 和 $Sname \rightarrow Sdept$ 都是直接函数依赖

之所以要区分完全FD与部分FD、直接FD与传递FD，是因为某些不好的部分FD或传递FD会带来过高的数据冗余，并因此产生插入、修改、删除等异常。

2.4 候选码

设 K 为 $R(U, F)$ 中的属性或属性组合。若 $K \xrightarrow{F} U$ ，则 K 称为 R 的一个**候选码**，简称**码**。

如果 $K \rightarrow U$ ，则 K 称为 R 的一个**超码**。

候选码是最小的超码，候选码的真子集一定不是超码。

候选码的超集是超码。

如果关系 R 的所有属性 U 是 R 的码，称为**全码**。

若关系模式 R 有多个候选码，则选定其中的一个做为主码。

□ [例6.2]

- $S(\underline{Sno}, Sdept, Sage)$ 中, 单个属性Sno是码(也是超码), $(Sno, Sdept)$ 是超码但不是码;
- $SC(\underline{Sno}, \underline{Cno}, Grade)$ 中, (Sno, Cno) 是码, $(Sno, Cno, Grade)$ 是超码但不是码, $(Sno, Grade)$ 既不是码也不是超码。

□ [例] Student(Sno, Sname, Ssex, Sage, Sdept)中

- 单个属性Sno是码, 单个属性Sname也是码(不允许学生同名)

□ [例6.3] 关系 $R(\text{演奏者}P, \text{作品}W, \text{听众}A)$ 中

- 一个演奏者可以演奏多个作品
- 某一作品可被多个演奏者演奏
- 听众可以欣赏不同演奏者的不同作品
- 关系 R 的码为 (P, W, A) , 即All-Key

包含在任何一个候选码中的属性, 称为**主属性**。

不包含在任何码中的属性称为**非主属性或非码属性**。

□ [例6.2]

- $S(\underline{Sno}, Sdept, Sage)$ 中, 只有一个候选码Sno, 所以: Sno是关系S的主属性, Sdept和Sage是关系S的2个非主属性;
- $SC(\underline{Sno}, \underline{Cno}, Grade)$ 中, 只有一个候选码 (Sno, Cno) , 所以: Sno和Cno是关系SC的2个主属性, Grade是关系SC的非主属性。

□ [例] Student(Sno, Sname, Ssex, Sage, Sdept)中

- 有两个候选码: Sno 和 Sname (不允许学生同名)
- 所以: Sno 和 Sname是关系Student的2个主属性, Ssex, Sage, Sdept是关系Student的3个非主属性

□ [例6.3] 关系 $R(\text{演奏者}P, \text{作品}W, \text{听众}A)$ 中

- 只有一个全码 (P, W, A)
- 所以: P, W, A 都是关系R的主属性, 在关系R中没有非主属性

2.5 外码

关系模式R中属性或属性组X并非R的码, 但X是另一个关系模式的码, 则称X是R的**外部码也称外码**。

主码与外码一起提供了表示关系间联系的手段

2.6 函数依赖的发现

函数依赖的**存在性判定**:

假设在关系模式R(U)上存在函数依赖: $X \rightarrow Y$

r是依据关系模式R(U)建立起来的任意一个关系, 那么关系r必满足:

从关系r中任取两个元组 r_1 和 r_2 , 如果元组 r_1 在X这组属性上的取值 $r_1[X]$ 等于元组 r_2 在X这组属性上的取值 $r_2[X]$, 即: $r_1[X] = r_2[X]$

则它们在Y属性组上的取值也必定相等, 即: $r_1[Y] = r_2[Y]$

- 特定的关系实例虽然不能用于函数依赖的发现, 但可以用于否定某些函数依赖

- 如何发现一个关系中的函数依赖:对关系中各个属性的语义进行分析,寻找它们相互之间存在的语义联系;也可以分析属性与属性之间在取值对应关系上的数量特征,进而判断它们之间是否存在某种函数依赖;属性之间的取值对应关系的数量特征有三种:**一对一;一对多;多对多**
- 函数依赖反映的是同一个关系中的两个属性子集之间在取值上的依存关系,这种依存关系实际上也是一种数据完整性约束.因此,我们也可以通过数据完整性约束的分析来寻找属性之间的函数依赖关系

2.7 2NF(排除部分依赖)

如果一个关系模式R的所有属性都是不可分的基本数据项,则 $R \in 1NF$

- 1NF是对关系模式的最起码的要求.不满足1NF的数据库模式不能称为关系数据库

若关系模式 $R \in 1NF$,并且**每一个非主属性都完全函数依赖于任何一个候选码**,则 $R \in 2NF$

推论: 候选码为单属性或者全码, 则属于2NF

- 在一个满足2NF的关系R中,不允许存在如下的函数依赖:

A是关系R的非主属性,K是关系R的候选码,且A部分函数依赖于K,即 $K \xrightarrow{P} A$

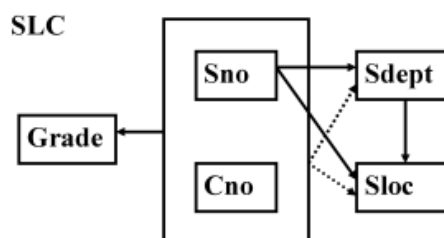
- $1NF \rightarrow 2NF$:消除非主属性对候选码的部分函数依赖, 把部分函数依赖投影出来单独成表

示例:

□ [例6.4] S-L-C(Sno,Sdept,Sloc,Cno,Grade), Sloc为学生的住处, 并且每个系的学生住在同一个地方。S-L-C的码为(Sno,Cno)。

➤ 函数依赖有

- $(Sno, Cno) \xrightarrow{F} Grade$
- $Sno \rightarrow Sdept, (Sno, Cno) \xrightarrow{P} Sdept$
- $Sno \rightarrow Sloc, (Sno, Cno) \xrightarrow{P} Sloc$
- $Sdept \rightarrow Sloc$



➤ 非主属性Sdept和Sloc部分函数依赖于码(Sno, Cno)

➤ 关系模式S-L-C不属于2NF

一个关系不属于2NF,会产生以下问题:

- 数据冗余
- 修改复杂,更新异常
- 插入异常
- 删除异常

产生原因:一个关系模式中有两类非主属性:一类对码是完全函数依赖,一类对码不是完全函数依赖.

解决方法:用**投影分解**把关系模式分解成两个关系模式.

2.8 3NF(排除传递依赖)

设关系模式 $R(U, F) \in 1NF$, 若 R 中不存在这样的码 X , 属性组 Y 及非主属性 $Z (Z \not\subseteq Y)$, 使得 $X \rightarrow Y, Y \rightarrow Z$ 成立, $Y \not\rightarrow X$, 则称 $R(U, F) \in 3NF$.

另一种定义:

若 R 满足 2NF, 且它的每一个非主属性都不传递依赖于任何候选码。

- $2NF \rightarrow 3NF$: 消除非主属性对键的传递函数依赖, 把传递函数依赖投影出来单独成表
- 缺点: 3NF 只限制了非主属性对键的依赖关系, 而没有限制主属性对键的依赖

2.9 BCNF(排除主属性对键的依赖)

BCNF(Boyce Codd Normal Form)是修正的 3NF.

设关系模式 $R(U, F) \in 1NF$, 若 $X \rightarrow Y$ 且 $Y \not\subseteq X$ 时 X 必含有码, 则 $R(U, F) \in BCNF$.

换言之, 在关系模式 $R(U, F)$ 中, 如果每一个决定因素都包含候选码, 则 $R \in BCNF$.

$3NF \rightarrow BCNF$: 消除原关系中主属性对键的部分函数依赖和传递函数依赖。

具有的性质:

- 所有非主属性都完全函数依赖于每个候选码
- 所有主属性都完全函数依赖于每个不包含它的候选码
- 没有任何属性完全函数依赖于非码的任何一组属性

如果一个关系数据库中的所有关系模式都属于 BCNF, 那么在函数依赖范畴内, 它已实现了模式的彻底分解, 达到了最高的规范化程度, 消除了插入异常和删除异常。

3NF 的不彻底性表现在可能存在主属性对码的部分依赖和传递依赖。

示例:

□ [例6.5] 考察关系模式 $C(Cno, Cname, Pcn)$

- 它只有一个码 Cno , 没有任何属性对 Cno 部分依赖或传递依赖, 所以 $C \in 3NF$.
- 同时 C 中 Cno 是唯一的决定因素, 所以 $C \in BCNF$.
- 对于关系模式 $SC(Sno, Cno, Grade)$ 可作同样分析。

□ [例6.6] 关系模式 $S(Sno, Sname, Sdept, Sage)$,

- 假定 $Sname$ 也具有唯一性, 那么 S 就有两个码, 这两个码都由单个属性组成, 彼此不相交。
- 其他属性不存在对码的传递依赖与部分依赖, 所以 $S \in 3NF$.
- 同时 S 中除 Sno , $Sname$ 外没有其他决定因素, 所以 $S \in BCNF$.

3.数据依赖的公理系统（Armstrong公理系统）

3.1 逻辑蕴涵

对于满足一组函数依赖 F 的关系模式 $R(U,F)$,其任何一个关系 r , 若函数依赖 $X \rightarrow Y$ 都成立, 则称 F 逻辑蕴涵 $X \rightarrow Y$ 。

可记为: $F \models X \rightarrow Y$

3.2 基本规则

设有关系模式 $R(U,F)$, U 是关系模式 R 的属性集总体, F 是 U 上的一组函数依赖。

- A1 自反律: 若 $Y \subseteq X \subseteq U$,则 $X \rightarrow Y$ 为 F 所蕴涵
- A2 增广律: 若 $X \rightarrow Y$ 为 F 所蕴涵, 且 $Z \subseteq U$,则 $XZ \rightarrow YZ$ 为 F 所蕴涵
- A3 传递律: 若 $X \rightarrow Y$ 及 $Y \rightarrow Z$ 为 F 所蕴涵, 则 $X \rightarrow Z$ 为 F 所蕴涵

使用自反律得到的函数依赖是平凡函数依赖

3.3 扩充规则

- 合并规则:若 $X \rightarrow Y$ 及 $X \rightarrow Z$,则 $X \rightarrow YZ$.
- 分解规则:若 $X \rightarrow Y$ 及 $Z \subseteq Y$,则 $X \rightarrow Z$.
- 伪传递规则:若 $X \rightarrow Y$ 及 $WY \rightarrow Z$,则 $XW \rightarrow Z$.

引理1: $X \rightarrow A_1A_2 \dots A_k$ 成立的充分必要条件是 $X \rightarrow A_i$ 成立. $(i = 1, 2, \dots, k)$

3.4 函数依赖集的闭包

在关系模式 $R(U,F)$ 中,为 F 所逻辑蕴涵的函数依赖的全体叫做 F 的闭包,记为 F^+ .也即:

$F^+ = \{X \rightarrow Y | F \models X \rightarrow Y\}$.

3.5 Armstrong 公理系统的有效性和完备性

$A1, A2, A3$ 称为Armstrong公理系统.

- 有效性:由 F 出发根据Armstrong公理推导出来的每一个函数依赖一定在闭包 F^+ 中
- 完备性:闭包 F^+ 中的每一个函数依赖,必定可以由 F 出发根据Armstrong公理推导出来

3.6 属性集的闭包

3.6.1 定义

设 F 为属性集 U 上的一组函数依赖, $X \subseteq U, X_F^+ = \{A | F \models X \rightarrow A \text{ 且 } A \in U\}, X_F^+$ 称为属性集 X 关于函数依赖集 F 的闭包.

这样判定一个函数依赖 $X \rightarrow Y$ 是否属于"由 F 根据Armstrong公理推导出来的函数依赖的集合",就可以转化成判断 $Y \subseteq X_F^+$ 是否成立.即如果 $Y \subseteq X_F^+$,则 $F \models X \rightarrow Y$ 成立.

3.6.2 引理2

设 F 为属性集 U 上的一组函数依赖, $X, Y \subseteq U, X \rightarrow Y$ 能由 F 根据Armstrong公理导出的充分必要条件是 $Y \subseteq X_F^+$.

3.6.3 求解 X_F^+ 的算法

算法6.1: 求属性集 $X (X \subseteq U)$ 关于 U 上的函数依赖集 F 的闭包 X_F^+

□ 输入: 函数依赖集 F , 属性集 X

□ 输出: 属性集闭包 X_F^+

- ① 令 $X^{(0)} = X, i = 0$
- ② 求 Y , 这里 $Y = \{A \mid (\exists V)(\exists W)(V \rightarrow W \in F \wedge V \subseteq X^{(i)} \wedge A \in W)\}$.
- ③ $X^{(i+1)} = Y \cup X^{(i)}$.
- ④ 判断 $X^{(i+1)} = X^{(i)}$ 是否成立?
- ⑤ 若 $X^{(i+1)} = X^{(i)}$ 成立 或 $X^{(i)} = U$ 成立, 则 $X^{(i)}$ 就是 X_F^+ , 算法终止。
- ⑥ 若否, 则 $i = i + 1$, 返回第②步。

属性集闭包的计算示例

□ [例6.11] 已知关系模式 $R(U, F)$, 其中: $U = \{A, B, C, D, E\}$,
 $F = \{AB \rightarrow C, B \rightarrow D, C \rightarrow E, EC \rightarrow B, AC \rightarrow B\}$. 求 $(AB)_F^+$.

解:

- $X^{(0)} = AB$
- 计算 $X^{(1)}$:
 - 逐一扫描集合 F 中的各个函数依赖, 寻找左部为 $X^{(0)}$ 的子集的函数依赖;
 - 得到两个函数依赖: $AB \rightarrow C, B \rightarrow D$. 于是: $Y = \{C, D\}$
 - 于是: $X^{(1)} = Y \cup X^{(0)} = \{A, B, C, D\}$
- 因为 $X^{(1)} \neq X^{(0)}$, 所以计算 $X^{(2)}$:
 - 逐一扫描集合 F 中的各个函数依赖, 寻找左部为 $X^{(1)}$ 的子集的函数依赖;
 - 得到两个函数依赖: $C \rightarrow E, AC \rightarrow B$. 于是: $Y = \{B, E\}$
 - 于是: $X^{(2)} = Y \cup X^{(1)} = \{A, B, C, D, E\}$
- 因为 $X^{(2)}$ 已等于全部属性集合 U , 所以 $(AB)_F^+ = X^{(2)} = \{A, B, C, D, E\}$.

3.7 函数依赖集的覆盖与等价

3.7.1 覆盖

设一个关系模式 R, F 和 G 是 R 上的两个函数依赖集, 如果 F 逻辑蕴涵 G 中所有函数依赖, 则称: F 覆盖 G , 即 $G \subseteq F^+$.

3.7.2 等价

如果两个函数依赖集的闭包相等(即 $F^+ = G^+$), 则称 F 和 G 等价.

3.7.3 引理3

$F^+ = G^+$ 的充分必要条件是 $F \subseteq G^+ \wedge G \subseteq F^+$.

3.8 极小函数依赖集

3.8.1 定义

如果函数依赖集F满足下列条件, 则称F为一个极小函数依赖集, 亦称为最小依赖集或最小覆盖。

- F中任一函数依赖的右部仅含有一个属性
- F中不存在这样的函数依赖 $X \rightarrow A$,使得F与 $F - \{X \rightarrow A\}$ 等价
- F中不存在这样的函数依赖 $X \rightarrow A$,X有真子集Z使得 $(F - \{X \rightarrow A\}) \cup \{Z \rightarrow A\}$ 与F等价

其中,条件1不是必须的,只是为了方便对条件2和3的检查

条件2表明,在极小函数依赖集F中,不允许存在冗余的函数依赖(可以由F中的其他函数依赖导出)

条件3表明,在极小函数依赖集F中,不允许存在部分函数依赖

3.8.2 定理3

每一个函数依赖集F均等价于一个极小函数依赖集 F_m ,此 F_m 称为F的最小依赖集

□ 证: 构造性证明, 分三步对F进行“极小化处理”, 找出F的一个最小依赖集。

① 逐一检查F中各函数依赖 $FD_i: X \rightarrow Y$, 若 $Y = A_1A_2...A_k$, $k \geq 2$, 则用 $\{X \rightarrow A_j | j = 1, 2, ..., k\}$ 来取代 $X \rightarrow Y$ 。

➤ 引理6.1保证了F变换前后的等价性

② 逐一检查F中各函数依赖 $FD_i: X \rightarrow A$, 令 $G = F - \{X \rightarrow A\}$, 若 $A \in X_G^+$, 则从F中去掉此函数依赖。

➤ 由于F与G等价的充要条件是 $A \in X_G^+$, 因此F变换前后是等价的。

③ 逐一取出F中各函数依赖 $FD_i: X \rightarrow A$, 设 $X = B_1B_2...B_m$, $m \geq 2$, 逐一考查 $B_i (i = 1, 2, ..., m)$, 若 $A \in (X - B_i)_F^+$, 则以 $(X - B_i) \rightarrow A$ 取代 $X \rightarrow A$ 。

➤ 由于F与 $F - \{X \rightarrow A\} \cup \{Z \rightarrow A\}$ 等价的充要条件是 $A \in Z_F^+$, 其中 $Z = (X - B_i)$ 。

□ 最后剩下的F就一定是极小依赖集。因为对F的每一次“改造”都保证了改造前后的两个函数依赖集等价, 因此剩下的F与原来的F等价。(证毕)

3.8.3 极小函数依赖集的不唯一

定理3的证明过程是求F极小函数依赖集的过程,也是检验F是否为极小函数依赖集的一个算法,若改造后的F与原来的F相同,说明F就是一个最小依赖集

示例:

□ [例6.13] $F = \{A \rightarrow B, B \rightarrow A, B \rightarrow C, A \rightarrow C, C \rightarrow A\}$

➤ F的最小依赖集: $F_{m1} = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$

➤ F的最小依赖集 F_m 不一定是唯一的, 它与对各函数依赖 FD_i 及 $X \rightarrow A$ 中X各属性的处置顺序有关。

➤ F_{m1} 、 F_{m2} 都是F的最小依赖集:

• $F_{m1} = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$

• $F_{m2} = \{A \rightarrow B, B \rightarrow A, A \rightarrow C, C \rightarrow A\}$

3.8.4 补充算法1：寻找与函数依赖集F等价的极小函数依赖集

算法核心思想:

- 消除F中的部分函数依赖
- 消除冗余的函数依赖(如果存在一个函数依赖 $X \rightarrow Y$, 能够被从F中的其他函数依赖中推导得到, 那么需要从F中剔除 $X \rightarrow Y$)

具体操作:

【补充算法】寻找与函数依赖集 F 等价的极小函数依赖集 G

1. 令 $G = F$
 - 将 G 中每一个形如 $X \rightarrow (A_1, A_2, \dots, A_n)$ 的函数依赖替换为如下一组依赖因素为单个属性的函数依赖: $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n$
2. 对 G 中的每一个函数依赖 $X \rightarrow A$ 作如下的处理:
 - 对决定因素 X 中的每一个属性 B 作如下处理:
 - 1) 计算属性集的闭包 $(X - B)_G^+$;
 - 2) 如果 $A \in (X - B)_G^+$, 则用新的函数依赖 $(X - B) \rightarrow A$ 替换原来的函数依赖 $X \rightarrow A$;
3. 对 G 中的每一个函数依赖 $X \rightarrow A$ 作如下处理:
 - 1) 令 $N = G - \{X \rightarrow A\}$;
 - 2) 计算属性集的闭包 X_N^+ ;
 - 3) 如果 $A \in X_N^+$, 那么从 G 中删去函数依赖 $X \rightarrow A$;
4. 将 G 中每一组形如 $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n$ (决定因素相同) 的函数依赖合并为一个函数依赖: $X \rightarrow (A_1, A_2, \dots, A_n)$

4.模式的分解

4.1 基本定义

关系模式 $R(U, F)$ 的一个分解是指:

$$\rho = \{R_1(U_1, F_1), R_2(U_2, F_2), \dots, R_n(U_n, F_n)\}$$

其中:

- $U = \bigcup_{i=1}^n U_i, \forall U_i \subseteq U_j, 1 \leq i, j \leq n$
- F_i 是 F 在属性集 U_i 上的投影, 也被称为“ F_i 是 F 在关系 R_i 上的投影”

投影定义:

与函数依赖集合 $\{X \rightarrow Y | X \rightarrow Y \in F^+ \wedge XY \subseteq U_i\}$ 等价的 F_i 叫做 F 在属性集 U_i 上的投影。

- 在一个关系模式 $R(U, F)$ 中, 可以用与 F 等价的依赖集 G 来取代 F

4.2 模式分解的三个定义

分解后的模式应与原模式等价, 等价意味着:

- 无损连接
- 保持函数依赖
- 既有无损连接, 又要保持函数依赖

模式分解的目标: 既有无损连接, 又要保持函数依赖

不同分解的示例：

□ [6.14] 关系模式 $R(\{Sno, Sdept, Mname\}, \{Sno \rightarrow Sdept, Sdept \rightarrow Mname\})$

□ 分解一：

$$\rho_1 = \{R_1(\{Sno\}, \emptyset), R_2(\{Sdept\}, \emptyset), R_3(\{Mname\}, \emptyset)\}$$

➤ 分解 ρ_1 既没有无损连接性，也不保持函数依赖；

□ 分解二：

$$\rho_2 = \{R_1(\{Sno, Sdept\}, \{Sno \rightarrow Sdept\}), R_2(\{Sno, Mname\}, \{Sno \rightarrow Mname\})\}$$

➤ 分解 ρ_2 具有无损连接性，但不保持函数依赖；

□ 分解三：

$$\rho_3 = \{R_1(\{Sno, Sdept\}, \{Sno \rightarrow Sdept\}), R_2(\{Sdept, Mname\}, \{Sdept \rightarrow Mname\})\}$$

➤ 分解 ρ_3 既具有无损连接性，又保持函数依赖。

Sno	Sdept	Mname
S1	D1	张五
S2	D1	张五
S3	D2	李四
S4	D3	王一

4.3 引理4

符号定义：

- 设 $\rho = \{R_1(U_1, F_1), R_2(U_2, F_2), \dots, R_k(U_k, F_k)\}$ 是 $R(U, F)$ 的一个分解， r 是 $R(U, F)$ 的一个关系
- $r_i = \pi_{R_i}(r) = \{t.U_i | t \in r\}$ 是关系 r 在关系模式 R_i 上的投影
- 定义 $m_\rho(r) = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_k}(r)$ ，即 $m_\rho(r)$ 是 r 在 ρ 中各关系模式上投影的连接

引理4：

设 $R(U, F)$ 是一个关系模式， $\rho = \{R_1(U_1, F_1), R_2(U_2, F_2), \dots, R_k(U_k, F_k)\}$ 是 R 的一个分解， r 是 R 的一个关系， $r_i = \pi_{R_i}(r)$ ，则：

1. $r \subseteq m_\rho(r)$
2. 若 $s = m_\rho(r)$ ，则 $\pi_{R_i}(s) = r_i$ ，即 $\pi_{R_i}(m_\rho(r)) = r_i$
3. $m_\rho(m_\rho(r)) = m_\rho(r)$

4.4 无损连接性

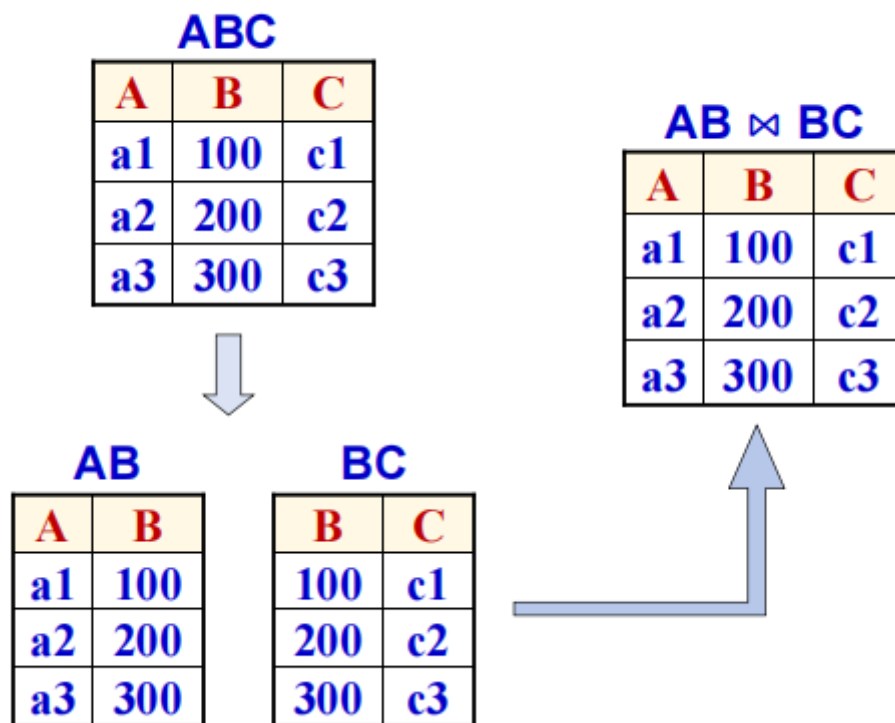
4.4.1 定义

设 $\rho = \{R_1(U_1, F_1), R_2(U_2, F_2), \dots, R_k(U_k, F_k)\}$ 是 $R(U, F)$ 的一个分解，若对 $R(U, F)$ 的任何一个关系 r 均有 $r = m_\rho(r)$ 成立，则称分解 ρ 具有无损连接性。简称 ρ 为无损连接。

- 对于不具备无损连接性的分解来说，关系 r 与分解后的子关系满足：

$$r \subset \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_k}(r)$$

一个具有‘无损连接性’的模式分解



4.4.2 判定

对于 $R(U, F)$ 的一个分解 $\rho = \{R_1(U_1, F_1), R_2(U_2, F_2)\}$, 如果 $(U_1 \cap U_2) \rightarrow (U_1 - U_2) \in F^+$ 或 $(U_1 \cap U_2) \rightarrow (U_2 - U_1) \in F^+$ 成立, 则分解 ρ 具有无损连接性。

无损连接判断

□ [思考] 设有一个关系模式 $T(A, B, C)$, 函数依赖集为 F , 请判断下述的分解 $\rho = \{T_1, T_2\}$ 是否具有无损连接性?

	函数依赖集 F	分解 ρ	是否满足无损连接性?
1	$\{A \rightarrow B\}$	$T_1(A, B) \quad T_2(A, C)$	yes
2	$\{A \rightarrow C, B \rightarrow C\}$	$T_1(A, B) \quad T_2(A, C)$	yes
3	$\{A \rightarrow B\}$	$T_1(A, B) \quad T_2(B, C)$	no
4	$\{A \rightarrow B, B \rightarrow C\}$	$T_1(A, C) \quad T_2(B, C)$	no

4.5 保持函数依赖

4.5.1 定义

若 $F^+ = (F_1 \cup F_2 \cup \dots \cup F_k)^+$, 则 $R(U, F)$ 的分解
 $\rho = \{R_1(U_1, F_1), R_2(U_2, F_2), \dots, R_k(U_k, F_k)\}$ 保持函数依赖

4.5.2 判定

只需判断 F 和 $(F_1 \cup F_2 \cup \dots \cup F_k)$ 是否等价

显然, $(F_1 \cup F_2 \cup \dots \cup F_k) \subseteq F^+$ 成立; 只需判断 $F \subseteq (F_1 \cup F_2 \cup \dots \cup F_k)^+$ 是否成立

4.6 模式分解的算法

- 若要求分解保持函数依赖, 那么模式分离总可以达到3NF, 但不一定能达到BCNF
- 若要求分解既保持函数依赖, 又具有无损连接性, 可以达到3NF, 但不一定能达到BCNF
- 若要求分解具有无损连接性, 那一定可达到4NF

4.6.1 补充算法2: 候选码的计算

Q: 如何寻找一个关系模式 $R(U, F)$ 的候选码?

- 方法1: 运用Armstrong公理系统做如下推导: $F \models K \xrightarrow{f} U$
- 方法2: 运用属性集闭包的概念, 寻找满足条件 $(K_F^+ == U)$ 的最小属性集合 K
- 方法3: 运用极小函数依赖集来优化方法2

定义:

设有关系模式 $R(U, F)$, U 是关系 R 的属性集合, F 是关系上的极小函数依赖集。根据 F 中的函数依赖, 可以将属性集合 U 划分为以下三个子集:

- 只在函数依赖的左边出现过的属性集合 U_L
- 只在函数依赖的右边出现过的属性集合 U_R
- 在两边都出现过的属性集合 U_A

其中:

- U_L 中的属性是每一个候选码的组成部分
- U_R 中的属性不可能出现在任何一个候选码中
- 在候选码计算中, 只需对 U_A 中的属性进行FOR循环检查

算法具体步骤:

输入: 关系 R 的属性集合 U , 极小函数依赖集 F
只在函数依赖的右边出现过的属性集合 U_R
在函数依赖的左右两边都出现过的属性集合 U_A
输出: 候选码 K

```
set  $K := U - U_R$  ;
for each attribute  $A$  in  $U_A$ 
{
    compute  $(K - A)_F^+$  ;
    if  $(K - A)_F^+$  contains all the attributes in  $R$ 
    then set  $K := K - A$  ;
}
return  $K$  ;
```

/ 对 U_A 中属性的不同处理顺序, 可能得到不同的计算结果 K */*

示例:

□ [例1] 寻找下述关系模式的候选码

$R(A, B, C) \quad F: \{A \rightarrow B, B \rightarrow A, A \rightarrow C\};$

解1: $K = \{A, B, C\}$

$\therefore (K-A)_F^+ = \{B, C\}_F^+ = \{A, B, C\} = U \quad \therefore K = K-A = \{B, C\}$

$\therefore (K-B)_F^+ = \{C\}_F^+ = \{C\} \neq U \quad \therefore$ 不能再从 $\{B, C\}$ 中去掉属性 B

$\therefore (K-C)_F^+ = \{B\}_F^+ = \{A, B, C\} = U \quad \therefore K = K-C = \{B\}$

最后得到该关系的一个候选码 $\{B\}$

解2: $K = \{A, B, C\}$

$\therefore (K-B)_F^+ = \{A, C\}_F^+ = \{A, B, C\} = U \quad \therefore K = K-B = \{A, C\}$

$\therefore (K-A)_F^+ = \{C\}_F^+ = \{C\} \neq U \quad \therefore$ 不能再从 $\{A, C\}$ 中去掉属性 A

$\therefore (K-C)_F^+ = \{A\}_F^+ = \{A, B, C\} = U \quad \therefore K = K-C = \{A\}$

最后得到该关系的另一个候选码 $\{A\}$

4.6.2 算法：转换为3NF既有无损连接性又保持函数依赖的分解

□ 输入：关系模式 $R(U, F)$

□ 输出：满足3NF且既有无损连接性又保持函数依赖的分解 ρ

- 1) 计算 F 的极小函数依赖集，并用来代替 F 进行后续的模式分解;
- 2) $\rho = \emptyset$; // 初始化分解 ρ 为空集
- 3) 对 F 中的每一个函数依赖 $X \rightarrow Y$ 做如下处理:
 - 如果在分解 ρ 中找不到满足下述条件的关系模式 $Z(U_Z, F_Z) : XY \subseteq U_Z$
 - 则由 X 和 Y 合并构成一个新的子关系模式并加入到分解 ρ 中;
- 4) 如果关系 R 的所有候选码都没有出现在分解 ρ 的关系模式中，即：找不到一个原关系 R 的候选码 K 和一个关系模式 $Z(U_Z, F_Z) (Z \in \rho)$ ，满足 $K \subseteq U_Z$
那么，就从关系 R 中任选一个候选码 K ，由 K 中的属性单独构成一个关系模式并加入到分解 ρ 中去。

4.6.3 算法：转换为BCNF的无损连接分解

□ 输入：关系模式 $R(U, F)$

□ 输出：到BCNF且具有无损连接性的分解 ρ

□ 算法：

- ① 令 $\rho = \{R(U, F)\}$
- ② 检查 ρ 中各关系模式是否均属于BCNF。若是，则算法终止。
- ③ 设 ρ 中 $R_i(U_i, F_i) \notin BCNF$ ，那么必有 $X \rightarrow A \in F_i^+ (A \notin X)$ ，且 X 非 R_i 的候选码。对 $R_i(U_i, F_i)$ 进行如下分解：
 - $\sigma = \{S_1, S_2\}, U_{S_1} = XA, U_{S_2} = U_i - A$
 - 以 σ 代替 ρ 中的 $R_i(U_i, F_i)$
 - 返回步骤②。

4.6.4 引理5

若 $\rho = \{R_1(U_1, F_1), R_2(U_2, F_2), \dots, R_k(U_k, F_k)\}$ 是 $R(U, F)$ 的一个无损连接分解。

$\sigma = \{S_1, S_2, \dots, S_m\}$ 是 ρ 中 $R_i(U_i, F_i)$ 的一个无损连接。那么

$\rho' = \{R_1, \dots, R_{i-1}, S_1, S_2, \dots, S_m, R_{i+1}, \dots, R_k\}$ 也是 $R(U, F)$ 的无损连接分解。

4.6.5 引理6

$$(R_1 \bowtie R_2) \bowtie R_3 = R_1 \bowtie (R_2 \bowtie R_3)$$

4.6.6 补：候选码的另一个算法

设关系模式 R , F 是 R 的函数依赖集，求候选码：

1. 列出 U_L, U_R, U_A, U_N 属性集包含的元素

- 在此之前可以先去求 U_L 属性的闭包，如果其包含 R 中所有元素，那么 U_L 属性对应的元素就是唯一候选码

2. 设 $X, Y \rightarrow X = U_L \cup U_N, Y = U_L \cup U_R \cup U_A$

3. 求 X 的闭包

4. 如果 X 的闭包包含 R 的全部属性元素， X 即为 R 的唯一候选码，结束。

若 X_F^+ 不全包含 R 的元素：

5. 从 Y 中依次取出一个元素，设该元素为 A ，求 $(XA)_F^+$ ，若取出元素和 X 组合求得的闭包包含关系模式中的所有元素，直到试完 Y 中全部的元素。

若 Y 中的元素取出与 X 组合均包含 R 全部属性，此时所有候选码被找出。

若 Y 中还有元素与 X 组合求得的闭包不包含全部元素，则从这些元素中依次取出两个开始继续与 X 组合。

参考：[候选码的计算](#)

4.6.7 补：分解为具有无损连接性和保持函数依赖的3NF的算法

输入：关系模式 $R(U, F)$

输出：具有无损连接性和保持函数依赖的3NF分解 $\rho = \{R_1, R_2, \dots, R_k\}$

算法：

1. 最小化：求 F 的最小函数依赖集 F_m

2. 排除：若 F_m 中存在 $X \rightarrow A$ ，使得 $XA = U$ ，则 R 已是3NF，转到6

3. 独立：若 R 中某些属性未出现在 F_m 中任一函数依赖的左部或右部，则将他们从 R 中分出去，单独构成一个关系子模式

4. 分组（相同左部原则）：对于 F_m 中的每一个 $X \rightarrow A$ ，都构成一个关系子模式 XA ，但若有 $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n$ ，可用合并规则合并为 $X \rightarrow A_1 A_2 \dots A_n$ 作为一个子模式。经过以上几步，求出函数依赖保持性分解： $\rho = \{R_1, R_2, \dots, R_k\}$

5. 添键：若 ρ 中没有子模式含有 R 的候选码 X ，则令 $\rho = \rho \cup \{X\}$ ；若存在 R_i 包含于 $R_j (i \neq j)$ ，则删去 R_i

6. 停止分解，输出 ρ

参考：[3NF的无损连接和保持函数依赖的算法](#)